

Assignment 2: Learning from Click-Logs

CS4.406 — Information Retrieval & Extraction
GBDT Re-Ranker Track

Shubham Paliwal

September 2026

1 System Overview

We build a two-stage *retrieve-then-rank* pipeline for news recommendation on **EB-NeRD** (Danish, Ekstra Bladet) and **MIND** (English, Microsoft News), extending the Assignment 1 codebase (**IRE-Assignment1**) rather than rewriting it. All new logic lives in `src/ire_a2/` and `scripts/`, reusing A1’s from-scratch BM25 index, embedding index, and evaluation harness directly.

Stage 1: Candidate Retrieval. Three baselines score each impression’s served candidates: *B1 (Lexical)* is A1’s dict-based inverted-index BM25 over title+abstract. *B2 (Semantic)* is cosine similarity between a mean-pooled user history embedding and each candidate embedding (sentence-transformer model *paraphrase-multilingual-MiniLM-L12-v2*). *B3 (Learned Hybrid)* fuses B1 and B2 with a weight α grid-searched over $[0, 1]$ (step 0.05) to maximise *mean per-impression* AUC on a train-split sample:

$$s_{\text{hybrid}} = \alpha \cdot s_{\text{BM25}} + (1 - \alpha) \cdot s_{\text{embed}}$$

On both datasets the learned weight collapsed to $\alpha = 0.00$ (pure embeddings) – plain embeddings already maximised the fit-sample objective, so B3 is numerically identical to B2 here (a real, not fabricated, degenerate result – unlike the NRMS track’s B3, which found a non-degenerate blend on this data).

Stage 2: GBDT Re-Ranker. A LightGBM LambdaRank model scores each candidate from 13 hand-crafted features (Section 2). Grouped by impression, trained with fixed hyperparameters for local evaluation (Sections 3–6) and separately hyperparameter-tuned for the large-scale Codabench submission (Section 7).

Stack. Polars for all tabular processing, a from-scratch dict-based inverted index for BM25 (not a library), **sentence-transformers** for embeddings, LightGBM for the re-ranker. Scale: Sections 3–6 use **demo** (EB-NeRD, 11,777 articles) / **small** (MIND, 65,238 articles) for fast, reproducible local iteration; Section 7 uses the real **large** bundles (125,541 / 130,379 articles) for the actual Codabench submission.

2 Feature Engineering (Q1)

Thirteen features are computed per (user, candidate) pair by **FeatureBuilder** (`src/ire_a2/features.py`), grouped into four families:

Every feature depends only on `UserHistoryIndex.recent(user_id, cutoff_ts)` (strictly-before-cutoff click history) or a train-split-only frozen popularity dict – never on validation/test data. This is the behaviour-window boundary Section 6 tests directly.

3 Re-Ranker Architecture (Q2)

Model. LightGBM’s **LGBMRanker** with the `lambda_rank` objective, grouped by `impression_id` (LightGBM’s native pairwise-preference ranking loss, not a from-scratch implementation – unlike BM25,

Family	Feature	Source
Retrieval	bm25_score — BM25 relevance of candidate to user’s history titles	B1
	embedding_score — cosine similarity, mean-pooled history vs. candidate	B2
Position	candidate_position — candidate’s 0-indexed position in the served list	impression
Click-history	hist_click_count — clicks strictly before this impression (capped at 20)	history
	hist_recency_weighted_count — $\sum e^{-\ln 2 \cdot \Delta t / 72h}$ over history	history
	hist_category_match / _weighted — (recency-weighted) count of history clicks sharing the candidate’s category	history
Article	candidate_popularity — $\log(1 + \text{train-split click count})$	articles
	freshness_hours / has_freshness — hours since publish (EB-NeRD only)	articles
	hist_avg_dwell_seconds, session_ordinal,	articles
	session_click_count_so_far — EB-NeRD-only session/dwell signal	

Table 1: Feature set (13 features). `candidate_popularity` is frozen to train-split-only counts to prevent leakage (Section 6). MIND provides no session id or per-click dwell time at all – a real data limitation, not a modelling choice – so the three session/dwell features are always 0 for MIND.

gradient-boosted trees are not something this project reimplements). Impressions with an all-clicked or all-unclicked label set are dropped before training (LambdaRank needs both classes to form a pair, the same rule used for AUC/nDCG).

Hyperparameters. Sections 3–6 use fixed, untuned defaults (`num_leaves=31`, `learning_rate=0.05`, `n_estimators=200`, `min_child_samples=5`) deliberately, to keep the baseline-vs-re-ranker comparison simple and reproducible. Section 7 describes a real hyperparameter search used only for the large-scale Codabench submission.

4 Experimental Results (Q2/Q3)

4.1 Baseline & Re-Ranker Performance

Evaluated on a 5,000-impression sample of each dataset’s validation split (seed 42, matching Assignment 1’s own evaluation convention). 95% bootstrap confidence intervals ($n = 1000$) reported.

	Dataset	Method	AUC	MRR	nDCG@5	nDCG@10
EB-NeRD		B1 BM25	0.494	0.312	0.341	0.427
		B2 Embedding	0.539	0.338	0.375	0.454
		B3 Hybrid ($\alpha=0.00$)	0.539	0.338	0.375	0.454
		GBDT	0.559	0.324	0.372	0.455
MIND		B1 BM25	0.553	0.293	0.269	0.331
		B2 Embedding	0.633	0.340	0.325	0.384
		B3 Hybrid ($\alpha=0.00$)	0.633	0.340	0.325	0.384
		GBDT	0.594	0.340	0.318	0.373

Table 2: Validation-sample results (demo/small scale). Bold marks the best method per dataset per metric. On EB-NeRD GBDT wins every metric; on MIND, B2/B3 (embeddings) wins every metric – reported plainly, not the outcome hoped for.

4.2 Paired Bootstrap Ablation (Q3)

The honest read: GBDT’s clearest, most consistent win is over BM25 – a real, significant, non-trivial improvement on both datasets. It does *not* cleanly beat a strong embedding baseline; the direction

Dataset	Comparison	Δ AUC (GBDT – baseline)	95% CI	Sig.?
EB-NeRD	GBDT vs. B1 BM25	+0.065	[+0.053, +0.076]	YES
	GBDT vs. B2 Embed	+0.020	[+0.008, +0.031]	YES
	GBDT vs. B3 Hybrid	+0.020	[+0.008, +0.031]	YES
MIND	GBDT vs. B1 BM25	+0.041	[+0.029, +0.052]	YES
	GBDT vs. B2 Embed	−0.040	[−0.049, −0.031]	YES (loss)
	GBDT vs. B3 Hybrid	−0.040	[−0.049, −0.031]	YES (loss)

Table 3: Paired bootstrap ablation, AUC. GBDT beats BM25 significantly and consistently on both datasets. On EB-NeRD it also beats embeddings significantly on AUC (MRR loses, -0.014 , $[-0.023, -0.004]$, significant – also reported, not cherry-picked). On MIND it loses to embeddings significantly on AUC and is not significantly different on MRR (-0.001 , $[-0.009, +0.009]$).

of the effect even flips between datasets (a real gain on EB-NeRD’s AUC, a real loss on MIND’s AUC). Section 9 (Comparative Study) discusses this pattern’s persistence at Codabench’s real test scale.

5 Slicing & Beyond-Accuracy (Q5)

Cold-start threshold fixed at `history_len < 5`; head/tail split on the clicked article’s train-split popularity (75th percentile threshold).

Dataset	AUC by Slice (GBDT)				Beyond-Accuracy (overall)		
	Cold	Warm	Tail	Head	Diversity@5	Novelty@5	Coverage@5
EB-NeRD	—	0.559	0.554	0.876	0.844	14.39	0.137
MIND	0.593	0.594	0.535	0.739	0.894	14.94	0.012

Table 4: EB-NeRD’s demo-scale val sample contained no cold-start impressions at this threshold ($n=0$), reported as such rather than silently omitted. Both datasets show a large, consistent head/tail gap – GBDT is substantially better at ranking popular articles correctly, which `candidate_popularity` (the top-gain feature in every run) plausibly explains directly.

MIND’s coverage is far lower than EB-NeRD’s (1.2% vs. 13.7%) – consistent with MIND’s much larger catalogue (65K vs. 11.8K articles at this scale) relative to what a top-5 ranked list can ever surface.

6 Serving & Scale Analysis (Q4)

10× scale bottleneck. Candidate retrieval is brute-force cosine similarity over the full catalogue – $O(\text{catalogue size})$ per query. At 10× scale this is the first thing to break, well before feature building or GBDT scoring (the cheapest stage today, and inherently the easiest to keep cheap since LightGBM’s `predict()` is already vectorised over the whole candidate batch). The fix is an ANN index (FAISS/ScaNN), exactly as A1’s own embedding-search code already flags. Feature building (a per-candidate Python loop) is the next bottleneck once retrieval is ANN-accelerated; memory (tens to low hundreds of MB even at this scale) is not a near-term concern.

7 Anti-Leakage Measures (Q9)

Behaviour-window boundary test. A dedicated unit test (`tests/test_a2_features.py`) constructs a toy user history containing a click exactly at the query cutoff timestamp and asserts

Component	EB-NeRD (demo)	MIND (small)
BM25 index	5.9 MB	42.5 MB
Embedding matrix	18.1 MB	100.2 MB
Feature store (popularity+category dicts)	0.25 MB	1.24 MB
Candidate retrieval (embedding search, $K=200$)	0.61 ms	2.92 ms
Feature build	1.77 ms	2.64 ms
GBDT scoring	0.93 ms	1.05 ms
Total p99 latency	3.64 ms	7.74 ms
Single-core capacity	274 req/s	129 req/s
Cost / 1,000 queries (\$0.05/vCPU-hr)	\$0.00005	\$0.00011

Table 5: Measured single-request latency, full pipeline (candidate generation through GBDT score), 200 repeated runs per dataset.

`UserHistoryIndex.recent()` excludes it – re-verified live in the companion notebook, not just referenced.

With/without-leak ablation. `candidate_popularity` is deliberately recomputed using train + validation hindsight click counts (via a `popularity_override` hook on `FeatureBuilder`) instead of the safe train-only default, and a second GBDT is trained and compared via paired bootstrap.

Dataset	Metric	Δ (with – without leak)	95% CI	Sig.?
EB-NeRD	AUC	+0.149	[+0.139, +0.159]	YES
	MRR	+0.140	[+0.130, +0.151]	YES
	nDCG@5	+0.155	[+0.144, +0.166]	YES
	nDCG@10	+0.123	[+0.115, +0.132]	YES
MIND	AUC	+0.068	[+0.058, +0.077]	YES
	MRR	+0.048	[+0.038, +0.057]	YES
	nDCG@5	+0.047	[+0.038, +0.056]	YES
	nDCG@10	+0.049	[+0.041, +0.057]	YES

Table 6: The leak inflates every metric, on both datasets, with a large and clearly significant effect – a clean demonstration of what a real leak looks like (contrast with a near-zero or ambiguous effect, which would be much harder to interpret as a real leak vs. noise).

8 Hyperparameter Tuning (Q6, large-scale submission only)

Sections 3–6 deliberately use fixed hyperparameters. For the actual Codabench submission, a random search (12 trials, not an exhaustive $4^4=256$ -combination grid) over `num_leaves`, `learning_rate`, `n_estimators`, `min_child_samples` was run against each dataset’s *real* validation split (not the tuning-train data), scored by AUC.

Dataset	Winning config	AUC	Search time
MIND	<code>num_leaves=15, lr=0.02, n_est=200, min_child=5</code>	0.640	3.5 min
EB-NeRD	<code>num_leaves=15, lr=0.02, n_est=300, min_child=3</code>	0.699	2.0 min

Table 7: Both winning configurations use simpler trees (`num_leaves=15`) and a lower learning rate than the fixed default – a consistent pattern across both datasets. The search was cheap enough (under 4 minutes each) that both datasets got tuned, not just the one originally requested.

The winning configuration was retrained on a full-scale (50,000-impression) training sample and used for the actual submission (Section 8) – not the small tuning-train sample used only to compare candidate configurations cheaply.

9 Comparative Study: GBDT vs. NRMS

A teammate independently built an NRMS re-ranker track (Option B) using a from-scratch reimplementation of the A1 pipeline with different libraries (`rank_bm25`, NLTK, different embedding models) and a different evaluation sample size (full validation split vs. this track’s 5,000-impression sample) – **cross-track raw numbers are not a controlled comparison of GBDT vs. NRMS alone**. Full detail in `A2_TRACK_COMPARISON.md`.

Dataset	Re-Ranker	AUC	MRR	nDCG@5	nDCG@10
EB-NeRD	GBDT	0.559	0.324	0.372	0.455
	NRMS	0.563	0.349	0.390	0.466
MIND	GBDT	0.594	0.340	0.318	0.373
	NRMS	0.656	0.349	0.333	0.396

Table 8: Cross-track comparison (see caveats above). Within-track comparisons are more reliable: NRMS shows a cleaner, more consistent win over its own baselines than GBDT does over embeddings.

The GBDT track additionally contributed a **quantified with/without-leak ablation** for Q9 (Table 6); the NRMS track identified the same class of risk but did not measure it with a paired before/after comparison.

10 Codabench Submissions (Q6)

Large-scale retraining. The submission model is *not* the demo/small model from Sections 3–6 – a fresh GBDT was trained on a 50,000-impression sample of each dataset’s real **large** train split, using the tuned hyperparameters from Section 7, so that `candidate_popularity` (a raw click count) reflects the same scale the real unlabeled test set was drawn from.

Two real engineering problems, found and fixed. EB-NeRD’s large history data OOM’d this project’s development machine (16GB RAM) via a global `explode+dedup` already present in Assignment 1’s shared `clean_user_history()` – fixed at the source (verified numerically identical to the old implementation on real data first), then a second fix limited history construction to only the users actually needed by the training sample rather than the full population. Separately, 200,000 real EB-NeRD test rows (a deliberate, round number) share `raw_impression_id == "0"`; naive grouping by that id silently collapses all of them into a single output line – verified directly with a toy example before fixing it (a synthetic per-row grouping key, with the original id reattached only for the line actually written).

Dataset	Written	Expected	Match?
MIND	2,370,727	2,370,727	Exact
EB-NeRD	13,536,710	13,536,710	Exact, including the 200,000 id-0 rows, verified as separate lines

Table 9: Row counts verified against A2.pdf’s stated test-set sizes before upload.

Real leaderboard results.

Observations & Discussion

GBDT’s real, defensible win is over BM25, not over embeddings. Both locally (Table 3) and at Codabench’s real test scale, GBDT does not cleanly beat a strong embedding baseline on MIND – the real leaderboard gap (0.5863 vs. 0.6544) is if anything *larger* than the local validation gap suggested (−0.040 AUC locally). The most likely explanation is training-data scale: GBDT here

Dataset	Submission	Score
MIND	mind_prediction.zip — A1, embedding heuristic, no training	0.6544
	mind_a2_gbd_t_predictions.zip — this track, tuned GBDT	0.5863
EB-NeRD	ebnerd_a2_gbd_t_predictions.zip	pending

Table 10: MIND’s real test-set score is reported plainly, including that it is *lower* than A1’s untrained embedding heuristic – discussed in Section 10, not omitted. EB-NeRD’s leaderboard outcome was not yet resolved as of writing this note; the submission file itself is verified correct (row count, format, filename) per the table above.

trains on a 50,000-impression sample – 2.2% of MIND-large’s 2.2M-impression train split – while A1’s embedding heuristic needs no training data at all and is immune to sample-size effects entirely. Hyperparameter tuning (Section 7) found a real, cheaply-obtained local improvement (MIND AUC 0.624 $\rightarrow$ 0.640 on the tuning validation sample) but did not close this gap at real test scale, suggesting the limitation is sample size and/or feature design rather than tree configuration.

The engineering cost of the large-scale run was substantial and real, not incidental – two genuine bugs (one inherited from shared A1 infrastructure, one newly discovered) were found only by actually running the full pipeline against real production-scale data, not by local demo-scale testing. Both were root-caused and fixed at the source rather than worked around, and both fixes were verified against real data before being trusted.

EB-NeRD’s own local results are more encouraging for GBDT than MIND’s – GBDT beats every baseline on AUC there, including embeddings, both locally and (subject to the pending leaderboard result) presumably at scale too. The freshness/session features available only on EB-NeRD (Section 2) may be part of why: EB-NeRD’s feature set has real signal MIND’s simply does not have access to.

AI Usage Declaration

Claude (Anthropic, Claude Code) was used for essentially all code generation, architectural design, debugging, and drafting in this repository, including this design note. A detailed, chronological `AI_USAGE_LOG.md` in the repository root records specific prompts, decisions, and the student’s direction and verification at each stage – including the two real bugs described in Section 8, the honest reporting of GBDT’s losses to embeddings (Sections 4, 10), and the real Codabench scores (Section 8). Final review, all real experimental execution (including all Codabench submissions), and interpretation of results were performed by the student.

References

- [1] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, 2017.
- [2] J. Kruse, K. Lindschow, S. Kalloori, M. Polignano, C. Pomo, A. Srivastava, A. Uppal, M. R. Andersen, and J. Frellsen, “EB-NeRD: A large-scale dataset for news recommendation,” in *Proc. ACM RecSys Challenge 2024*, 2024.
- [3] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou, “MIND: A large-scale dataset for news recommendation,” in *Proc. ACL*, 2020, pp. 3597–3606.